

LABSHEET 3

Step 1. Import the required libraries and load a grayscale image for processing.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files

# Upload image
uploaded = files.upload()

# Get the uploaded filename
filename = next(iter(uploaded))

# Load the image in grayscale
img = cv2.imread(filename, cv2.IMREAD_GRAYSCALE)

# Check whether image is loaded
if img is None:
    print("Error: Image could not be loaded.")
else:
    print("Image loaded successfully.")
    print("Image shape:", img.shape)

# Display the grayscale image
plt.figure(figsize=(6, 4))
plt.imshow(img, cmap="gray")
plt.title("Original Grayscale Image")
plt.axis("off")
plt.show()

print("Aryan Agarwal")
print("CU240251800")
```

Choose Files No file chosen

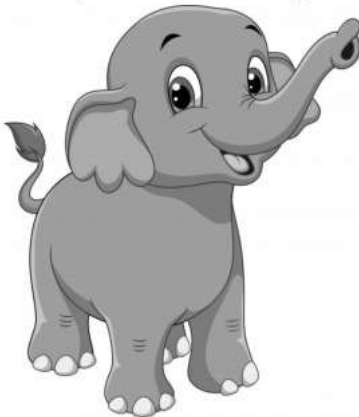
Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving e3ff3e7fd8db6509fbf20aa74220b3b4.jpg to e3ff3e7fd8db6509fbf20aa74220b3b4.jpg

Image loaded successfully.

Image shape: (724, 626)

Original Grayscale Image



Aryan Agarwal

Step 2. Display the original image and analyze its visual characteristics.

```
import cv2
import matplotlib.pyplot as plt
from google.colab import files

uploaded = files.upload()
img = cv2.imread(next(iter(uploaded)))
```

```
plt.figure(figsize=(4,3))
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.title("Original Image")
plt.axis("off")
plt.show()
plt.close()

print("The image contains different intensity levels, edges, textures and details.")

print("Name: Aryan Agarwal")
print("CU240251800")
```

Choose Files 63df92e5cc...5bc1e93.jpg
63df92e5cca1c49a3415891995bc1e93.jpg(image/jpeg) - 26133 bytes, last modified: 8/19/2026 - 100% done
 Saving 63df92e5cca1c49a3415891995bc1e93.jpg to 63df92e5cca1c49a3415891995bc1e93.jpg

Original Image



The image contains different intensity levels, edges, textures and details.
 Name: Aryan Agarwal
 CU240251800

✓ Step 3. Apply Gaussian Blur to reduce image noise while preserving important structures.

```
blur = cv2.GaussianBlur(img, (5,5), 0)

plt.figure(figsize=(4,3))
plt.imshow(cv2.cvtColor(blur, cv2.COLOR_BGR2RGB))
plt.title("Gaussian Blur")
plt.axis("off")
plt.show()
plt.close()

print("Gaussian Blur reduces noise while preserving important structures.")

print("Name: Aryan Agarwal")
print("CU240251800")
```

Gaussian Blur



Gaussian Blur reduces noise while preserving important structures.
 Name: Aryan Agarwal
 CU240251800

✓ Step 4. Apply Median Filtering to remove impulse (salt-and-pepper) noise from the image.

```
median = cv2.medianBlur(img, 5)
```

```
plt.figure(figsize=(4,3))
plt.imshow(cv2.cvtColor(median, cv2.COLOR_BGR2RGB))
plt.title("Median Filter")
plt.axis("off")
plt.show()
plt.close()

print("Median filtering is effective for removing salt-and-pepper noise.")

print("Name: Aryan Agarwal")
print("CU240251800")
```

Median Filter



Median filtering is effective for removing salt-and-pepper noise.
 Name: Aryan Agarwal
 CU240251800

Step 5. Apply Average (Mean) Filtering and compare its smoothing performance with Gaussian filtering.

```
mean = cv2.blur(img, (5,5))

plt.figure(figsize=(4,3))
plt.imshow(cv2.cvtColor(mean, cv2.COLOR_BGR2RGB))
plt.title("Mean Filter")
plt.axis("off")
plt.show()
plt.close()

print("Mean filtering provides strong smoothing but may blur edges.")
print("Gaussian filtering generally preserves edges better.")

print("Name: Aryan Agarwal")
print("CU240251800")
```

Mean Filter



Mean filtering provides strong smoothing but may blur edges.
 Gaussian filtering generally preserves edges better.
 Name: Aryan Agarwal
 CU240251800

Step 6. Implement Laplacian Filtering to enhance image edges and fine details

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

```

laplacian = cv2.Laplacian(gray, cv2.CV_64F)

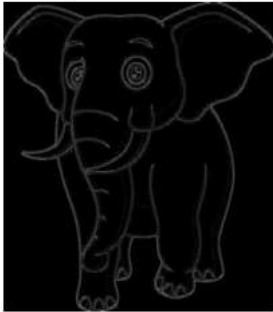
plt.figure(figsize=(4,3))
plt.imshow(abs(laplacian), cmap="gray")
plt.title("Laplacian Filtering")
plt.axis("off")
plt.show()
plt.close()

print("Laplacian filtering highlights edges and fine image details.")

print("Name: Aryan Agarwal")
print("CU240251800")

```

Laplacian Filtering



Laplacian filtering highlights edges and fine image details.
 Name: Aryan Agarwal
 CU240251800

Step 7. Apply Sobel Edge Detection in both horizontal and vertical directions to detect image gradients.

```

sobel_x = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
sobel_y = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)

plt.figure(figsize=(4,3))
plt.imshow(abs(sobel_x), cmap="gray")
plt.title("Sobel X - Horizontal")
plt.axis("off")
plt.show()
plt.close()

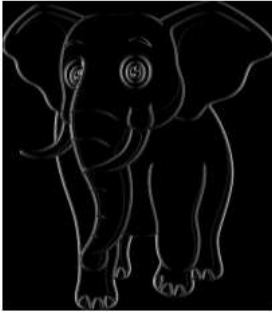
plt.figure(figsize=(4,3))
plt.imshow(abs(sobel_y), cmap="gray")
plt.title("Sobel Y - Vertical")
plt.axis("off")
plt.show()
plt.close()

print("Sobel X detects horizontal gradients.")
print("Sobel Y detects vertical gradients.")

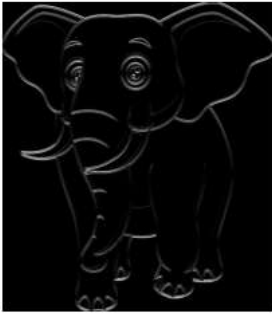
print("Name: Aryan Agarwal")
print("CU240251800")

```

Sobel X - Horizontal



Sobel Y - Vertical



Sobel X detects horizontal gradients.
 Sobel Y detects vertical gradients.
 Name: Aryan Agarwal
 CU240251800

Step 8. Compare the outputs of different low-pass and high-pass filters in terms of noise reduction, edge preservation, and computational performance.

```
print("Gaussian: Good noise reduction and better edge preservation.")
print("Median: Best for salt-and-pepper noise.")
print("Mean: Simple and fast but blurs edges.")
print("Laplacian: Enhances edges and fine details.")
print("Sobel: Detects horizontal and vertical edges efficiently.")
```

```
print("Name: Aryan Agarwal")
print("CU240251800")
```

Gaussian: Good noise reduction and better edge preservation.
 Median: Best for salt-and-pepper noise.
 Mean: Simple and fast but blurs edges.
 Laplacian: Enhances edges and fine details.
 Sobel: Detects horizontal and vertical edges efficiently.
 Name: Aryan Agarwal
 CU240251800

Step 9. Display the original and filtered images along with suitable observations highlighting the strengths and limitations of each filtering technique.

```
print("Low-pass filters reduce noise and smooth the image.")
print("Median filter is effective for salt-and-pepper noise.")
print("Gaussian filter provides smooth noise reduction.")
print("Mean filter is simple but may blur edges.")
print("Laplacian and Sobel are high-pass filters used for edge detection.")
```

```
print("Name: Aryan Agarwal")
print("CU240251800")
```

Low-pass filters reduce noise and smooth the image.
 Median filter is effective for salt-and-pepper noise.
 Gaussian filter provides smooth noise reduction.
 Mean filter is simple but may blur edges.
 Laplacian and Sobel are high-pass filters used for edge detection.

Name: Aryan Agarwal
CU240251800

Step 10. Interpret the results and discuss the applications of spatial filtering in real-world computer vision systems.

```
import cv2
import matplotlib.pyplot as plt
from google.colab import files

uploaded = files.upload()
filename = next(iter(uploaded))

img = cv2.imread(filename)

if img is None:
    print("Error: Image could not be loaded.")
else:
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    gaussian = cv2.GaussianBlur(gray, (5, 5), 0)
    median = cv2.medianBlur(gray, 5)
    mean = cv2.blur(gray, (5, 5))
    laplacian = cv2.convertScaleAbs(
        cv2.Laplacian(gray, cv2.CV_64F)
    )
    sobel_x = cv2.convertScaleAbs(
        cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
    )

    plt.figure(figsize=(8, 6))

    plt.subplot(2, 3, 1)
    plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
    plt.title("Original")
    plt.axis("off")

    plt.subplot(2, 3, 2)
    plt.imshow(gaussian, cmap="gray")
    plt.title("Gaussian")
    plt.axis("off")

    plt.subplot(2, 3, 3)
    plt.imshow(median, cmap="gray")
    plt.title("Median")
    plt.axis("off")

    plt.subplot(2, 3, 4)
    plt.imshow(mean, cmap="gray")
    plt.title("Mean")
    plt.axis("off")

    plt.subplot(2, 3, 5)
    plt.imshow(laplacian, cmap="gray")
    plt.title("Laplacian")
    plt.axis("off")

    plt.subplot(2, 3, 6)
    plt.imshow(sobel_x, cmap="gray")
    plt.title("Sobel X")
    plt.axis("off")

    plt.tight_layout()
    plt.show()
    plt.close()

    print("Observations:")
    print("Gaussian: Smooths noise with good edge preservation.")
    print("Median: Effective for salt-and-pepper noise.")
    print("Mean: Simple smoothing but may blur edges.")
    print("Laplacian: Enhances edges and fine details.")
    print("Sobel: Detects image gradients and edges.")

print("Name: Aryan Agarwal")
```

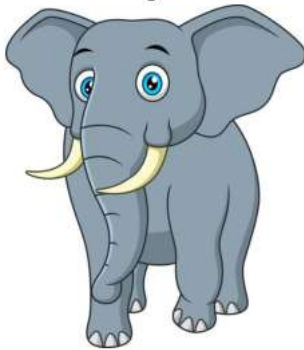
```
print('CU240251800')
```

Choose Files 63df92e5cc...5bc1e93.jpg

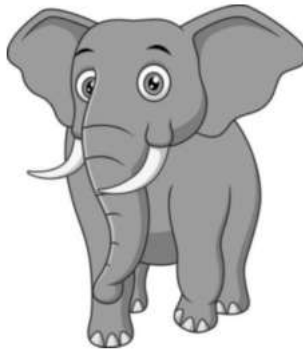
63df92e5cca1c49a3415891995bc1e93.jpg(image/jpeg) - 26133 bytes, last modified: 8/19/2026 - 100% done

Saving 63df92e5cca1c49a3415891995bc1e93.jpg to 63df92e5cca1c49a3415891995bc1e93 (1).jpg

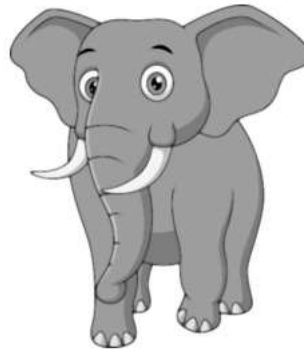
Original



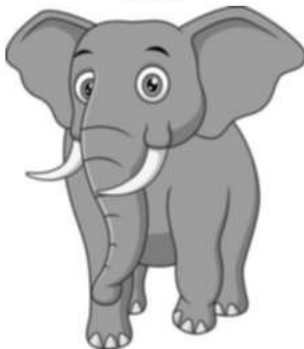
Gaussian



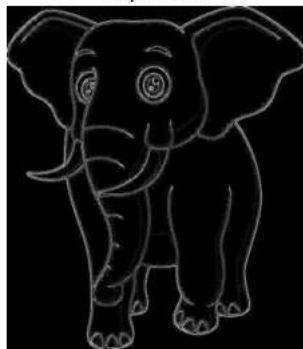
Median



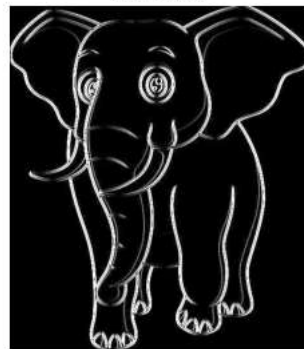
Mean



Laplacian



Sobel X



Observations:

Gaussian: Smooths noise with good edge preservation.

Median: Effective for salt-and-pepper noise.

Mean: Simple smoothing but may blur edges.

Laplacian: Enhances edges and fine details.

Sobel: Detects image gradients and edges.

Name: Aryan Agarwal

CU240251800